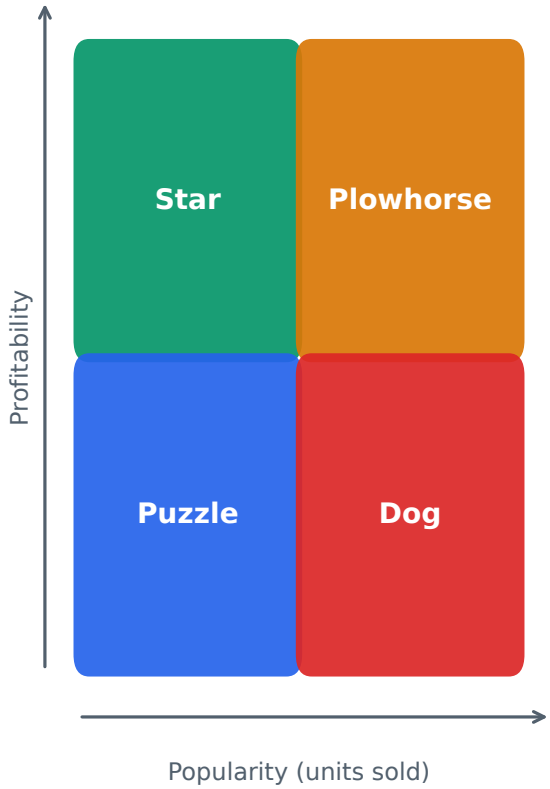
MenuMind

A Quantitative Methodology for Menu Profitability and Engineering Classification

How item-level profit is computed and how each dish is classified as a Star, Plowhorse, Puzzle, or Dog.



Technical Methodology Report

Derived directly from the production analytics engine.

Version 1.0

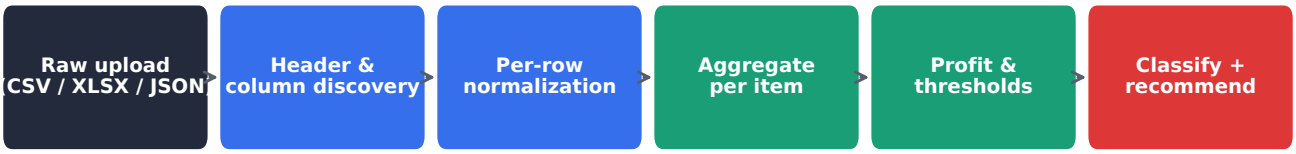
June 2026

1. Overview

MenuMind turns a raw restaurant sales export into an item-level profitability map. The system never asks the operator to pre-compute anything: it ingests whatever spreadsheet the point-of-sale produced, infers which columns mean what, reduces every transaction to four numbers per dish, and then places each dish into one of four strategic categories using the classical menu-engineering matrix of Kasavana & Smith (1982).

This report documents the complete chain of logic — from a messy CSV row to a Star/Plowhorse/Puzzle/Dog verdict and an actionable recommendation — exactly as implemented in the codebase. Every formula below corresponds to a concrete line in the analytics engine.

Processing pipeline at a glance



The four numbers that drive everything

After ingestion, each individual sales row is stored as exactly four analytic fields. Every downstream metric is built from these alone:

- item_name — the dish, cleaned and title-cased so that ' MARGHERITA pizza' and 'Margherita Pizza' aggregate together.
- quantity — integer units sold in that row (defaults to 1 if the export has no quantity column).
- revenue — money taken for that row, in INR. If only a unit price is present, revenue is reconstructed as price × quantity.
- unit_cost — cost of goods (COGS) to produce one unit; defaults to 0 when the export carries no cost column.

> *Design choice: revenue is stored as a row total, while cost is stored per unit. Section 4 shows how the engine reconciles the two so that profit is always computed on a per-unit basis.*

2. How Items Are Identified and Listed

Restaurant exports are notoriously messy: title rows, merged cells, blank lines, currency symbols, and inconsistent headers. The ingestion layer is built to survive all of this without manual cleanup.

2.1 Column discovery

The engine does not trust column positions. Instead it scores headers against a keyword dictionary and maps each physical column to one of five logical roles. A header (or, if the first row is junk, the first row that scores ≥ 2 keyword hits) is promoted to the schema.

Logical field	Matched against (sample aliases)
item_name	item, name, product, menu, dish, sku, description
quantity	qty, quantity, count, sold, units, orders, volume
revenue	revenue, sales, amount, total, gross, net, turnover, value
unit_cost	cost, cogs, expense, purchase, ingredient
date	date, time, day, businessdate, orderdate

Fallback heuristics handle exports with no usable headers. The item column is chosen as the most text-like column (low numeric ratio, moderate length, many distinct values); quantity is inferred as the low-magnitude numeric column; revenue as the highest-summing numeric column. This means even an unlabeled table still classifies correctly.

2.2 Per-row normalization and the validity gate

Every candidate row is cleaned and then must pass a strict gate before it is allowed to influence any metric. A row is KEPT only if all three hold:

- item_name is not junk (not blank, 'nan', 'total', 'subtotal', 'summary', etc.);
- revenue > 0;
- quantity > 0.

Rows failing the gate are counted as rows_rejected and reported back to the user, but never enter the profit math. This is what keeps summary rows and refunds from polluting the averages that decide classification.

> *Net is intentionally excluded from the revenue keyword used to pick the price fallback, so a 'net' column never masquerades as unit price. Numbers are parsed by stripping every character except digits, minus, and decimal point — so 'Rs 4,000' becomes 4000 cleanly.*

3. The Profit Model

All profitability is computed per menu item after aggregating every kept row for that item across the selected date range and restaurant. For an item i, the engine first sums the raw quantities, revenues, and costs:

$$Q_i = \sum q, \quad R_i = \sum r, \quad C_i = \sum (\text{unit\_cost} \times q)$$

Note that total cost is the sum of unit_cost × quantity over the rows — cost is weighted by how many units each row sold, while revenue is already a row total. From these three sums, the engine derives per-unit economics:

$$\bar{p}_i = \frac{R_i}{Q_i} \text{ (avg unit price)}, \quad \bar{c}_i = \frac{C_i}{Q_i} \text{ (avg unit cost)}$$

3.1 Profit per unit — the core metric

The single most important quantity in the whole system is the average profit margin earned on one unit of the item:

$$\text{profit}_i \;=\; \bar{p}_i - \bar{c}_i \;=\; \frac{R_i}{Q_i} - \frac{C_i}{Q_i}$$

This per-unit profit — not total profit — is the value plotted on the profitability axis of the menu matrix and the number shown on every card in the dashboard ('Unit Profit: Rs ...'). Using a per-unit figure makes a low-volume, high-margin dish directly comparable to a high-volume, thin-margin dish.

3.2 Gross profit contribution

For ranking which items actually move the bottom line, the engine also computes each item's total contribution to gross profit, and sums these for the business-wide figure reported in Insights:

$$\text{GP}_i = \text{profit}_i \times Q_i, \qquad \text{GP}_{\text{total}} = \sum_i \text{GP}_i$$

> *When an export carries no cost column, unit_cost defaults to 0, so profit collapses to average unit price and 'profit' should be read as gross revenue per unit. Supplying COGS is what turns the same pipeline into a true net-margin engine.*

4. Classifying Each Item

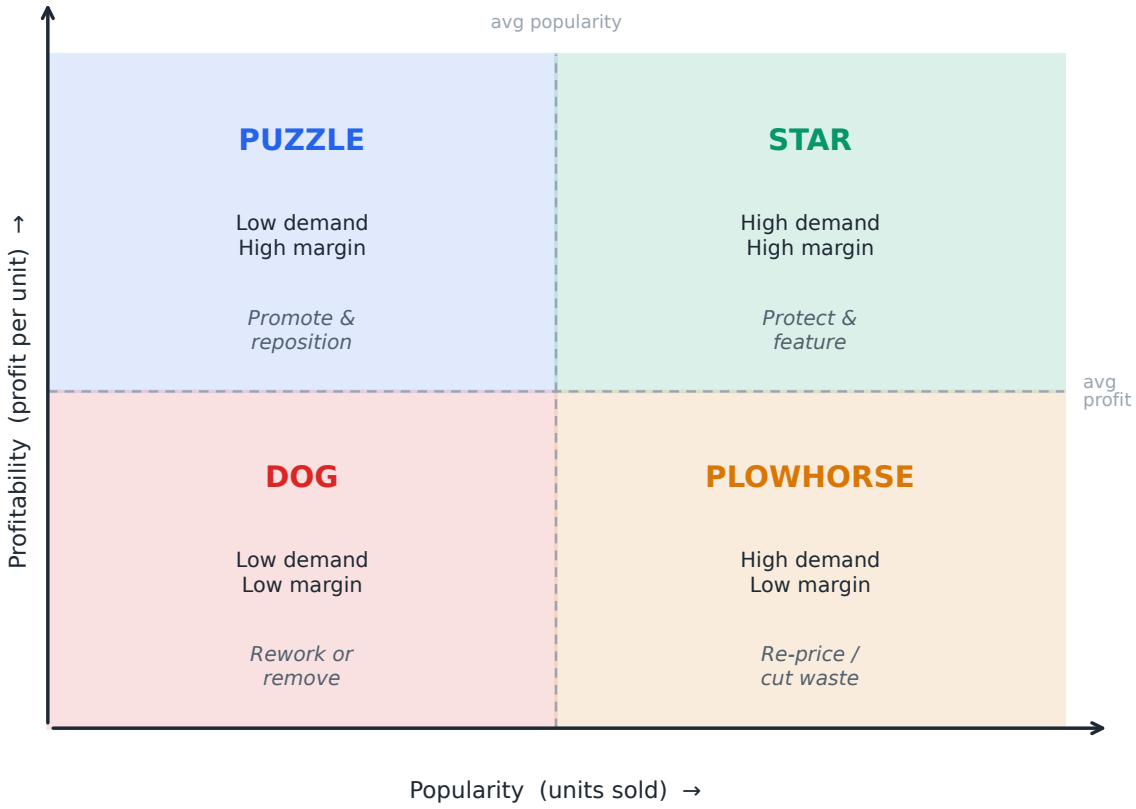
With a per-unit profit and a unit count for every item, classification is a relative comparison against the menu's own averages. The engine computes two thresholds across the N valid items:

$$\overline{\text{Pop}} = \frac{1}{N}\sum_i Q_i, \qquad \overline{\text{Prof}} = \frac{1}{N}\sum_i \text{profit}_i$$

Each item is then tagged on two binary axes — is it at or above the average on popularity, and at or above the average on profitability? The four combinations give the four classic categories:

- Star — popularity ≥ avg AND profit ≥ avg.
- Plowhorse — popularity ≥ avg, profit < avg.
- Puzzle — popularity < avg, profit ≥ avg.
- Dog — popularity < avg, profit < avg.

> *The comparison is inclusive (≥): an item sitting exactly on the average is given the benefit of the doubt and counts as 'high'. Thresholds are recomputed for every query, so categories always reflect the current filter window, not a fixed historical baseline.*



5. A Worked Example

To make the logic concrete, consider a small menu of four items over a reporting window. The table shows the aggregated sums the engine reads from the database, followed by the derived per-unit profit.

Item	Q (units)	R (Rs)	C total (Rs)	Profit/unit	Class
Margherita Pizza	320	224,000	89,600	420.0	Star
Garlic Bread	410	102,500	73,800	70.0	Plow
Truffle Risotto	70	98,000	42,000	800.0	Puzzle
House Salad	90	31,500	21,600	110.0	Dog

Step-by-step for Margherita Pizza

Average unit price = R / Q = 224000 / 320 = Rs 700.0
Average unit cost = C / Q = 89600 / 320 = Rs 280.0
Profit per unit = 700.0 − 280.0 = Rs 420.0

Applying the thresholds

Average popularity across the four items = 222 units.
Average profit per unit across the four items = Rs 350.0.

Margherita Pizza sells 320 units (≥ 222 → high popularity) and earns Rs 420/unit (≥ Rs 350 → high profitability). Both axes high → it is a STAR. Garlic Bread is popular but thin-margin → PLOWHORSE; Truffle Risotto is rich-margin but slow → PUZZLE; House Salad is low on both → DOG.

6. From Category to Action

Classification is only useful if it produces a decision. The recommendation engine maps each category to a deterministic, rule-based action, a justification, a priority, and a confidence score. The mapping is fixed (not model-generated), which makes every suggestion auditable and reproducible.

Star — priority High · confidence 0.92

Action: Protect quality; feature in high-visibility menu positions.

Why: High demand and high per-unit profit.

Plowhorse — priority High · confidence 0.88

Action: Test a small price increase, cut waste, or redesign portions.

Why: High demand but profit below the menu average.

Puzzle — priority Medium · confidence 0.84

Action: Improve placement, naming, photography, staff prompts before discounting.

Why: Healthy profit but demand below average.

Dog — priority Low · confidence 0.82

Action: Remove, rework, or bundle unless it has strategic value.

Why: Low demand and low profit.

7. Business-Wide Insights

Alongside per-item output, the engine summarizes the whole menu: total revenue, total units, and total gross profit ($\Sigma\ GP_i$); the revenue leader and the strongest profit contributor ($\max GP_i$); a margin watchlist of the three lowest profit-per-unit items; the category mix counts; and a simple revenue trend.

$$\text{trend} = \text{up if } R_{\text{last day}} > 1.1\,R_{\text{first day}};\ \text{down if } < 0.9\,R_{\text{first day}};\ \text{else stable}$$

8. Assumptions and Limitations

- Relative, not absolute — categories are defined against the menu's own averages. Every item could be profitable in absolute terms and half will still be labelled 'low profit'. The matrix ranks within a menu; it does not certify viability.
- Average-based thresholds — a single very-high-volume item drags the popularity average upward and can push moderate sellers below the line. Some practitioners prefer a 70%-of-average popularity rule or a median; this engine uses the plain mean for transparency.
- Cost data quality — profit accuracy is bounded by COGS accuracy. With no cost column, unit_cost = 0 and 'profit' equals unit price. Fixed costs, labour, and wastage are out of scope — this is a contribution-margin model, not full cost accounting.
- Per-unit averaging — price and cost are averaged over the window, so promotions, combos, and mid-period price changes are blended into a single effective margin per item.
- Window sensitivity — thresholds are recomputed per query, so the same dish can change category as the date filter changes. This is intended (it reflects current performance) but means categories are not permanent labels.

9. Conclusion

MenuMind reduces an arbitrary sales export to four trustworthy numbers per dish, computes a per-unit contribution margin, and benchmarks every item against the menu's own popularity and profitability averages to assign one of four strategic categories. Each category carries a fixed, auditable recommendation. The pipeline is deliberately transparent: there is no black box between the raw upload and the Star/Plowhorse/Puzzle/Dog verdict — only the explicit arithmetic documented here.

Symbol reference

Q_i — total units of item i · R_i — total revenue of item i
C_i — total cost (Σ unit_cost $\times$ q) · $\bar{p}_i$ — avg unit price = R_i/Q_i
$\bar{c}_i$ — avg unit cost = C_i/Q_i · $profit_i$ — per-unit profit = $\bar{p}_i - \bar{c}_i$
GP_i — gross profit contribution = $profit_i \times Q_i$
Pop, Prof bars — menu-wide averages used as classification thresholds